

Stack & Queues

Stack: Stack is a data structure which holds a certain type of data structure which LIFO principle for insertion and deletion. (It has functions like push, pop, top, size)

Queue: Queue is a data structure which holds a certain type of data structure which follows FIFO mechanism. (It has functions like push, pop, top, size)

```
class > > mp > > {  
  
    top = -1, int st[10];  
  
    push (n)  
    {  
        if (top >= 10) →  
            top = top + 1  
            st[top] = n;  
    }  
}
```

```
int top ()  
{  
    if (top == -1) →  
        return st[top];  
}
```

```
pop ()  
{  
    if (top == -1) —  
        top = top - 1;  
}
```

```
size ()  
{  
    return top + 1;  
}
```

```
class Q {
```

```
    size = 10, q[size], curSize = 0, start = -1, end = -1
```

```
    push (n)  
    {
```

```
        if (curSize == size) —
```

```
        if (curSize == 0)  
        {
```

```
            start = 0, end = 0
```

```

    }
    else
        end = (end + 1) % size;

    q[end] = n, curSize += 1;
}

```

```

pop()
{
    if (curSize == 0) ×
        el = q[start] ←
    if (curSize == 1)
        start = end = -1
    else
        start = (start + 1) % size;

    curSize -= 1;
    return el;
}

```

```

top()
{
    if (curSize == 0) —
        return q[start];
}

```

Ques.

20. Valid Parentheses

Solved

Easy Topics Companies Hint

Given a string `s` containing just the characters `'('`, `'{'`, `'['`, `')'`, `'}'` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: `s = "()"`

Output: `true`

Example 2:

Input: `s = "([)]"`

C++ Auto

```
1 class Solution {
2 public:
3     bool isValid(string s) {
4         int n = s.length();
5         stack<int> st;
6         for(int i = 0; i < n; i++){
7             if(s[i]=='(' || s[i]=='{' || s[i]=='['){
8                 st.push(s[i]);
9             }
10            else{
11                if(st.size() > 0){
12                    char top_char = st.top();
13                    if(top_char=='(' && s[i]==')' || top_char=='{' && s[i]=='}' || top_char=='[' && s[i]==']'){
14                        st.pop();
15                    }
16                    else{
17                        return false;
18                    }
19                }
20                else{
21                    return false;
22                }
23            }
24        }
25        return st.empty();
26    }
27 }
```

Prefix, Infix and Postfix Conversion -

Priority order -

↑
*
+
-
↓ decreasing priority.

equal priority -

Infix $\rightarrow (p+q)*(r)$

prefix $\rightarrow +pq(r)$

postfix $\rightarrow pqr+$ (stack based calculators used in)

i) Infix to postfix-

a + b * (c ^ d - e)
 ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑

★ for closing bracket pop everything till you get opening bracket.

i = 0	st	ans
a		a
+	+	a
b	+	ab
*	+ *	ab
(	+ * (	ab
c	+ * (	abc
^	+ * (^	abc
d	+ * (^	abcd
-	+ * (-	abcd ^
e	+ * (-	abcd ^ e

) + *
 (Remove opening bracket & also don't push that closing bracket).

abcd ^ e - * +

```

int Postfix (s)
{

```

```

    i = 0 , st , ans = ""

```

```

    while (i < n)
    {

```

```

        if ((s[i] >= 'A' && s[i] <= 'Z') ||
            (s[i] >= 'a' && s[i] <= 'z') ||
            (s[i] >= '0' && s[i] <= '9'))
            ans = ans + s[i];
    }
}

```



```

else if (s[i] == '(')
    st.push(s[i]);

else if (s[i] == ')')
{
    while (!st.empty() && st.top() != '(')
    {
        ans += st.top();
        st.pop();
    }
}

```

```

else
{
    while (!st.empty() && priority priority(s[i]) <= prio(st.top()))
    {
        ans += st.top();
        st.pop();
    }

    st.push(s[i]);
}

```

```

i++;
}

while (!st.empty())
{
    ans = ans + st.top();
    st.pop();
}

return ans;

```

2) Infix to postfix -

- Step-1. Reverse the infix (Closing bracket will be convert to opening & vice versa)
- Step-2. Convert into postfix
- Step-3. Reverse the answer.

Ex. Infix = $(A+B)*C-D+F.$

Step-1 $F + D - C * (B + A)^*$

Step 2	i	Stack	Ans
	F	—	F
	+	+	F
	D	+	F D
	-	+ -	F D
	C	+ -	F D C
	*	+ - *	F D C.
	(	+ - * (	F D C.
	B	+ - * C	F D C B
	+	+ - * (+	F D C B
	A	+ - * (+	F D C B A
	)	+ - *	F D C B A +
	.	,	F D C B A + * - +

in postfix conversion we would have first + popped (as the pop happens when priority is $(\leq)$ but here it is $(<)$)

Just push the opening bracket.

Step 3 . $+ - * + A B C D F$

```
string infixToPrefix(string s) {  
    stack<char> st;  
    string result = "";  
  
    // scan from right to left  
    for (int i = s.length() - 1; i >= 0; i--) {  
        char c = s[i];  
  
        if (isdigit(c)) {  
            result += c;  
        }  
        else if (c == ')') {  
            st.push(c);  
        }  
        else if (c == '(') {  
            while (!st.empty() && st.top() != ')') {  
                result += st.top();  
                st.pop();  
            }  
  
            // pop ')'   
            if (!st.empty()) st.pop();  
        }  
    }  
}
```

```

        else if (isOperator(c)) {
            while (!st.empty() && isOperator(st.top()) &&
                (precedence(st.top()) > precedence(c) ||
                 (precedence(st.top()) == precedence(c) &&
                  isRightAssociative(c)))) {
                result += st.top();
                st.pop();
            }
            st.push(c);
        }
    }

    // pop remaining operators
    while (!st.empty()) {
        result += st.top();
        st.pop();
    }

    // reverse at the end to get correct prefix
    reverse(result.begin(), result.end());
    return result;
}

```

Postfix to Infix → $AB - DE + F * /$

Rules →

- Push the operand into the stack.
- Whenever an operator comes pop second top then first top from the stack and keep the operator in between them and close them with the brackets and then push the whole element with bracket together, they will be counted as a single unit.

	<u>Ans</u>	<u>Stack</u>
...		A
A		AB
B	$(A - B)$	$(A - B)$
-		$(A - B), D$
D		$(A - B), D, E$
E		$(A - B), (D + E)$
+	$(D + E)$	
F	$(F * (D + E))$	$(A - B), (D + E), F$
*	$(A - B) / ((D + E) * F)$	$(A - B), (F * (D + E))$
/	$(A - B) / ((D + E) * F)$	$((A - B) / ((D + E) * F))$

↑ 2nd top ↑ first top → infix.

```

postfixToInfix (s)
{

```

```

    i = 0, st

```

```

    while (i < n)
    {

```

```

        if (operand)
        {

```

```

            st.push (s[i]);

```

```

        }

```

```

    else
    {

```

```

        t1 = st.top() st.pop()

```

```

        t2 = st.top() st.pop()

```

```

        con = '(' + t2 + s[i] + t1 + ')'

```

```

        st.push (con)

```

```

    }

```

```

    i++

```

```

}
return st.top();

```

→ Adding two strings takes additional time equal to length of strings.

Prefix to Infix Expression -

- 1) Iterate the prefix expression from the back and whenever we get an operand put it into the stack. When we get an operator take the first top then operator then the second top embrace them with a brackets and put all these as a single unit into the stack.

Rule prefix =

Ex. ++pq-m

↓
m
↓
p
↓
q
↓
+
↓
+
↓
+

Stack

m

m, p

(m-p)

(m-p), q

(m-p), q, p

(m-p), (p-q)

```

postfixToPrefix(s)
{

```

```

    i = n-1, st

```

```

    while (i >= 0)
    {

```

```

        if (operand) st.push(s[i])

```

```

    else
    {

```

```


```

```

        t1 = st.top st.pop

```

```

        t2 = st.top st.pop

```

```

        con = '(' + t1 + s[i] + t2 + ')'

```

```

        st.push(con)

```

```

    }

```

```

    i--

```

```

}
return st.top()

```

```

}

```

Postfix to Prefix -

Rule

- 1) Iterate from the front, whenever there is operand put it into the stack, when an operator comes, first take the operator then second top of the stack and then first top of the stack, this time don't surround these into brackets, now put this string into the stack.

(treat the obtained string as a single unit).

Ex postfix = $AB - DE + F * /$

i

A

B

-

D

E

+

F

*

- AB

+ DE

* + DE F

Stack

A

A, B

- AB

- AB, D

- AB, D, E

- AB, + DE

- AB, + DE, F

- AB, * + DEF

```

postfix ← prefix (s)
{
    i = 0 , st

    while (i < n)
    {
        if (operand) st.push (s[i])
        else
        {
            t1 = st.top    st.pop
            t2 = st.top    st.pop

```

```

            st.push (s[i] + t2 + t1)
            i++
        }
    }
    return st.top

```

Prefix to Postfix

- 1) Iterate from the back, whenever you see a operand push it into the stack , when we get a operator, take the top1 then top2 then operator and put this entire string into the stack as a single unit without any brackets.
- 2)

Ex. Prefix = $1 - AB * + DEF$

i
 F
 E
 D
 +
 *
 B
 A
 -
 /

DE+
 DE+F*
 AB-
 AB-DE+F*/

stack
 F
 F, E
 F, E, D
 F, DE+
 DE+F*
 DE+F*, B
 DE+F*, B, A
 DE+F*, AB-
 AB-DE+F*/.
 ↪ Postfix.

```

preIn To Postfix (s)
{
    i = n - 1    st
    while (i >= 0)
    {
        if (operand) st.push(s[i]);

```

```

        else
        {
            t1 = st.top    st.pop
            t2 = st.top    st.pop

            st.push ( t1 + t2 + s[i] );
        }
        i --
    }
    return st.top();
}

```

Implement Min Stack :- (to get minimum element from the stack).

Min Stack

→ push 12
 → push 15
 → push 10
 → getMin
 pop
 top
 getMin

value pushed → minimum till now.
 (12, 12) (15, 12) (10, 10)

pair < int, int >
 ↑ ↑
 val min


```

class st
{
    stack < pair < int, int > > st;

    push (val)
    {
        if (st.empty())
            st.push ({val, val});
        else
            st.push ({val, min (val, st.top().second)});
    }
}

```

```

getMin ()
{
    return st.top().second;
}

top ()
{
    return st.top().first;
}

```

Another method - Take mini variable, when we get new minimum, don't push it into the stack, instead push

$$\left(\underset{\substack{\text{(current that} \\ \text{was to be pushed)}}}{2 \times \text{val}} - \text{previous_min} \right) = \underset{\substack{\text{(it will be pushed)} \\ \text{(modified)}}}{\text{new_val}}$$

and store the 'val' in the mini_index

$$\begin{aligned}
 \text{as, } & \text{val} < \text{previous_min} \\
 & \text{val} + \text{val} < \text{previous_min} + \text{val} \\
 & 2 \times \text{val} - \text{previous_min} < \text{val} \\
 & \boxed{\text{new_val} < \text{val}}
 \end{aligned}$$

→ When top() is called, if mini > st.top(), means at st.top(), we have modified value, then return 'mini' for st.top(); else return st.top().

155. Min Stack

Solved

Medium

Topics

Companies

Hint

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the `MinStack` class:

- `MinStack()` initializes the stack object.
- `void push(int val)` pushes the element `val` onto the stack.
- `void pop()` removes the element on the top of the stack.
- `int top()` gets the top element of the stack.
- `int getMin()` retrieves the minimum element in the stack.

You must implement a solution with $O(1)$ time complexity for each function.

C++ Auto

```
1 class MinStack {
2 public:
3     stack<long long>st;
4     long long mini = LONG_LONG_MAX;
5     MinStack() {
6     }
7
8     void push(int val) {
9         if(st.empty()){
10             st.push(val);
11             mini = val;
12         }
13         else if(val>mini){
14             st.push(val);
15         }
16         else{
17             long long prev_mini = mini;
18             long long new_val = 2LL*val-prev_mini;
19             mini = val;
20             st.push(new_val);
21         }
22     }
```

```
24 void pop() {
25     long long value_to_be_deleted = st.top();
26     if(value_to_be_deleted>mini){
27         st.pop();
28     }
29     else{
30         mini = 2*mini-value_to_be_deleted;
31         st.pop();
32     }
33 }
34
35 int top() {
36     long long topi = st.top();
37     if(topi<mini){
38         return mini;
39     }
40     else{
41         return st.top();
42     }
43 }
44
45 int getMin() {
46     return mini;
47 }
48 };
49
```

Next Greater Element :-

Ex. arr = [6, 0, 8, 1, 3]
(next greater element) Ans = [8, 8, -1, 3, -1]

Traverse from the back , for the rightmost element put -1 in the answer array, put the element in the stack, for the next element if it is smaller than the st.top(), then it's next greater is st.top() , if it is greater than st.top(), pop until we get a element greater than it, then push it into the stack.

```
3     vector<int> nextGreaterElements(vector<int>& nums) {
4         int n = nums.size();
5         int last = nums[n-1];
6         int last_greater = -1;
7         for(int i = 0; i<n; i++){
8             if(nums[i]>last){
9                 last_greater = nums[i];
10                break;
11            }
12        }
13        stack<int>st;
14        vector<int>ans(n,-1);
15        ans[n-1] = last_greater;
16        st.push(nums[n-1]);
17        for(int i = n-2; i>=0; i--){
18            if(nums[i]<st.top()){
19                ans[i] = st.top();
20                st.push(nums[i]);
21            }
22            else{
23                while(!st.empty() && st.top()<nums[i]){
24                    st.pop();
25                }
26                if(!st.empty()){
27                    ans[i] = st.top();
28                }
29                else{
30                    ans[i] = -1;
31                }
32                st.push(nums[i]);
33            }
34        }
35        return ans;
36    }
37 };
```

Ques. Next Greater Element - II

503. Next Greater Element II

Attempted

Medium

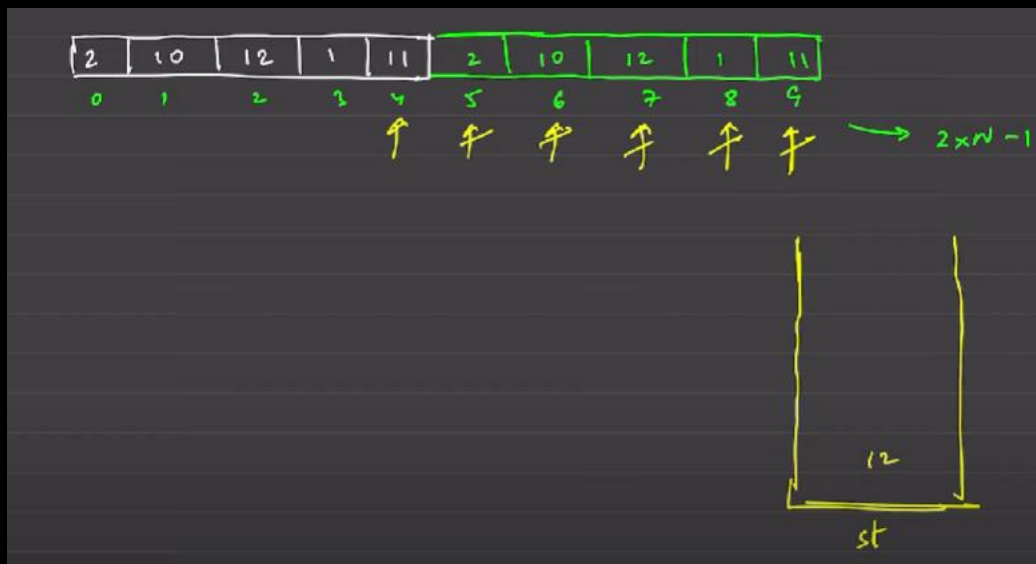
Topics

Companies

Given a circular integer array `nums` (i.e., the next element of `nums[nums.length - 1]` is `nums[0]`), return the **next greater number** for every element in `nums`.

The **next greater number** of a number `x` is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return `-1` for this number.

Extend the array to its double size, then push the elements in the stack, from the back if the upcoming element is greater than the top element in the stack, then pop the elements until we get a greater element. Then push the element. When we reach the index equal to $n-1$, store the next greater element as done in the previous question using an array.



```

1  class Solution {
2  public:
3      vector<int> nextGreaterElements(vector<int>& nums) {
4          int n = nums.size();
5          stack<int> st;
6          vector<int> ans(n, -1);
7          for(int i = 2*n-1; i>=n; i--){
8              int idx = i%n;
9              if(st.empty()){
10                 st.push(nums[idx]);
11             }
12             else if(st.top()>nums[idx]){
13                 st.push(nums[idx]);
14             }
15             else{
16                 while(!st.empty() && st.top()<=nums[idx]){
17                     st.pop();
18                 }
19                 st.push(nums[idx]);
20             }
21         }
22         for(int i = n-1; i>=0; i--){
23             if(nums[i]<st.top()){
24                 ans[i] = st.top();
25                 st.push(nums[i]);

```

```

26         }
27         else{
28             while(!st.empty() && st.top()<=nums[i]){
29                 st.pop();
30             }
31             if(!st.empty()){
32                 ans[i] = st.top();
33             }
34             else{
35                 ans[i] = -1;
36             }
37             st.push(nums[i]);
38         }
39     }
40     return ans;
41 }
42 };

```

Ques. Previous Smaller Element—

Similar to next greater element, in this iterate from the left, for the 0th index the ans will be -1. If the coming element is greater than st.top() then it's previous smaller element is st.top(), push that element in the stack, If the coming element is lesser than st.top(), then pop() elements from the stack until we get st.top()<arr[i], if !st.empty() then ans for that element will be st.top(), else -1. Push that element in the stack.

Smaller on Left

Difficulty: Medium Accuracy: 54.85% Submissions: 77K+ Points: 4 Average Time: 15m

Given an array `arr[]` of integers, for each element in the array, find the **nearest smaller element on its left**. If there is no such smaller element, return **-1** for that position.

Input: `arr[] = [1, 6, 2]`

Output: `[-1, 1, 1]`

Explanation:

There is no number to the left of 1, so we return -1.

After that, the number is 6, and the nearest smaller number on its left is 1.

Next, the number is 2, and the nearest smaller number on the left is also 1.

Input: `arr[] = [1, 5, 0, 3, 4, 5]`

Output: `[-1, 1, -1, 0, 3, 4]`

Explanation:

There is no number to the left of 1, so we return -1.

```

1 class Solution {
2 public:
3     vector<int> leftSmaller(vector<int> arr) {
4         // code here
5         int n = arr.size();
6         vector<int> ans(n, -1);
7         stack<int> st;
8         st.push(arr[0]);
9         ans[0] = -1;
10        for(int i = 1; i < n; i++){
11            if(arr[i] > st.top()){
12                ans[i] = st.top();
13                st.push(arr[i]);
14            }
15            else{
16                while(!st.empty() && arr[i] <= st.top()){
17                    st.pop();
18                }
19                if(!st.empty()){
20                    ans[i] = st.top();
21                }
22                st.push(arr[i]);
23            }
24        }
25        return ans;
26    }
27 };

```

Trapping Rainwater -



$$\sum_{i=0}^{n-1} \min(\text{leftMax}, \text{rightMax}) - \text{arr}[i]$$

total = 0

for (i = 0 → n-1)

if (arr[i] < leftMax && arr[i] < rightMax)

total += min(leftMax, rightMax) - arr[i];

return total

(width is 1 for a bar)

For left max and right max we will use prefix max and suffix max.

$prefix_Max[N]$
 $arr = [2, 1, 0, 5, 3]$
 $(prefix_max = [2, 2, 2, 5, 5])$
 $prefix_max[0] = arr[0]$
 $for (i = 1 \rightarrow n-1)$
 $\{$
 $prefix_max[i] = \max(prefix_max[i-1], arr[i])$
 $\}$

$suffix_max[N]$
 $arr = [2, 1, 0, 5, 3]$
 $suffix_max = [5, 5, 5, 5, 3]$
 $suffix_max[n-1] = arr[n-1]$
 $for (i = n-1 \rightarrow 0)$
 $\{$
 $suffix_max[i] = \max(suffix_max[i+1], arr[i])$
 $\}$

iterate from this side.

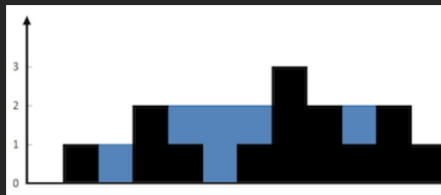
42. Trapping Rain Water

Solved

Hard Topics Companies

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:



Input: height = [0,1,0,2,1,0,1,3,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

C++ Auto

```

1 class Solution {
2 public:
3     int trap(vector<int>& height) {
4         int n = height.size();
5         vector<int> prefix_max(n);
6         vector<int> suffix_max(n);
7         prefix_max[0] = height[0];
8         for(int i = 1; i < n; i++){
9             prefix_max[i] = max(prefix_max[i-1], height[i]);
10        }
11        // Suffix_Max---->
12        suffix_max[n-1] = height[n-1];
13        for(int i = n-2; i >= 0; i--){
14            suffix_max[i] = max(suffix_max[i+1], height[i]);
15        }
16        // Total_water logged calculations---->
17        int total_water = 0;
18        for(int i = 1; i < n-1; i++){
19            if(prefix_max[i] > height[i] && suffix_max[i] > height[i]){
20                total_water += min(prefix_max[i], suffix_max[i]) - height[i];
21            }
22        }
23        return total_water;
24    }
25 };

```

Saved

Ln 20, Col

Approach 2-

```
int findTotal ( arr )
```

```
{  
    lMax = rMax = total = 0 , l = 0 , r = n-1
```

```
    while ( l < r )
```

```
    {
```

```
        if ( arr[l] <= arr[r] )
```


rmax l

```
        {  
            if ( lMax > arr[l] )
```

```
                total += lMax - arr[l]
```

```
            else
```

```
                lMax = arr[l]
```

```
            l = l + 1
```

```
        }
```

```
    else {
```

```
        if ( arr[r] < rmax ) {
```

```
            total += rmax - arr[r]
```

```
        }
```

```
    else {
```

```
        rmax = arr[r]
```

```
    }
```

```
    r = r + 1;
```

```
    }
```

```
}
```

```
return total;
```


r rmax

Sum of Subarray's minimum-

907. Sum of Subarray Minimums

Medium

Topics

Companies

Given an array of integers `arr`, find the sum of $\min(b)$, where `b` ranges over every (contiguous) subarray of `arr`. Since the answer may be large, return the answer **modulo** $10^9 + 7$.

Sum of subarray minimums

`arr = [3, 1, 2, 4]`

minimum of subarrays

`3` → `3` → `1` → `2` → `4` → `4`
`3, 1` → `1` → `1, 2` → `1` → `2, 4` → `2`
`3, 1, 2` → `1` → `1, 2, 4` → `1`
`3, 1, 2, 4` → `1`

Ex- $\overbrace{1, 4, 6, 7, 3, 7, 8, 1}^{4 \text{ } 3}$ $\Rightarrow$ 3 will be counted $4 \times 3 = 12$ times
 Now, for the above problem we need to find that how many times each element came in the subarrays as minimum element, If we take element '3' then it has come $4 \times 3 = 12$ times, so it will contribute $12 \times 3 = 36$ in the total sum.

`Arr = [1, 4, 6, 7, 3, 7, 8, 1]`
`index = 0 1 2 3 4 5 6 7`

index of previous smaller or equal $\rightarrow$ total element in between $= 4 - 0 = 4$
 index of next smaller $= 7$
 $\Rightarrow$ total element in between $= 7 - 4 = 3$

So, No. of subarrays, in which '3' contributed as smallest element $= 4 \times 3 = 12$.

IF previous_small equal doesn't exist then put it -1, if next_small don't exist put it N.

```

1  class Solution {
2  public:
3      int sumSubarrayMins(vector<int>& arr) {
4          int n = arr.size();
5          vector<int>prev_small(n);
6          vector<int>next_small(n);
7          stack<int>st;
8          prev_small[0] = -1;
9          st.push(0);
10         // prev_small array----->
11         for(int i = 1; i<n; i++){
12             if(arr[i]>=arr[st.top()]){
13                 prev_small[i] = st.top();
14             }
15             else{
16                 while(!st.empty() && arr[st.top()]>arr[i]){
17                     st.pop();
18                 }
19                 if(!st.empty()){
20                     prev_small[i] = st.top();
21                 }
22                 else{
23                     prev_small[i] = -1;
24                 }
25             }
26             st.push(i);
27         }

```

different from next small.

ind 0 1
arr = 1, 1
prev_small = -1, 0
next_small = 2, 2
Subarray → 1 → 1
1, 1 → 2
Sum due to 1 = 3

```

28         // for next_small array----->
29         stack<int>s;
30         next_small[n-1] = n;
31         s.push(n-1);
32         for(int i = n-2; i>=0; i--){
33             if(arr[i]>arr[s.top()]){
34                 next_small[i] = s.top();
35             }
36             else{
37                 while(!s.empty() && arr[s.top()]>=arr[i]){
38                     s.pop();
39                 }
40                 if(!s.empty()){
41                     next_small[i] = s.top();
42                 }
43                 else{
44                     next_small[i] = n;
45                 }
46             }
47             s.push(i);
48         }
49         const int MOD = 1e9 + 7;
50         long long total = 0;
51         for (int i = 0; i < n; i++) {
52             long long left = i - prev_small[i];
53             long long right = next_small[i] - i;
54             total = (total + (1LL * arr[i] * left * right) % MOD) % MOD;
55         }
56         return total;

```

Ques

Sum of subarray ranges →

Sum of subarray ranges → largest - smallest

arr = [1 4 3 2]

1		= 0		4		= 0		3		= 0		2		= 0
1	4	= 3		4	3	= 1		3	2	= 1				
1	4	3	= 3		4	3	2	= 2						
1	4	3	2	= 3										

So, sum of subarray ranges = sum of subarray maximum - sum of subarray minimum.

Similar to subarray min

For sum of subarray maximum also while abstracting prev_large or Equal, if prev_large don't exist put it -1 and if next_large don't exist put it n. Note the we are calculating prev_large or equal and next_large similar to prev question.

Code for sum of subarray maximum

```
57 long long sumSubarraysmax(vector<int>&arr){
58     int n = arr.size();
59     vector<int>prev_large(n);
60     vector<int>next_large(n);
61     stack<int>st;
62     prev_large[0] = -1;
63     st.push(0);
64     // prev_large array----->
65     for(int i = 1; i < n; i++){
66         if(arr[i] <= arr[st.top()]){
67             prev_large[i] = st.top();
68         }
69         else{
70             while(!st.empty() && arr[st.top()] < arr[i]){
71                 st.pop();
72             }
73             if(!st.empty()){
74                 prev_large[i] = st.top();
75             }
76             else{
77                 prev_large[i] = -1;
78             }
79         }
80         st.push(i);
81     }
82     // for next_small array----->
83     stack<int>s;
```

for previous_large calculate larger or equal.

```

84     next_large[n-1] = n;
85     s.push(n-1);
86     for(int i = n-2; i>=0; i--){
87         if(arr[i]<arr[s.top()]){
88             next_large[i] = s.top();
89         }
90         else{
91             while(!s.empty() && arr[s.top()]<=arr[i]){
92                 s.pop();
93             }
94             if(!s.empty()){
95                 next_large[i] = s.top();
96             }
97             else{
98                 next_large[i] = n;
99             }
100        }
101        s.push(i);
102    }
103    long long total = 0;
104    for (int i = 0; i < n; i++) {
105        long long left = i - prev_large[i];
106        long long right = next_large[i] - i;
107        total = (total + (1LL * arr[i] * left * right) );
108    }
109    return total;
110 }

```

Ques. Asteroid Collision-

735. Asteroid Collision

Medium

Topics

Companies

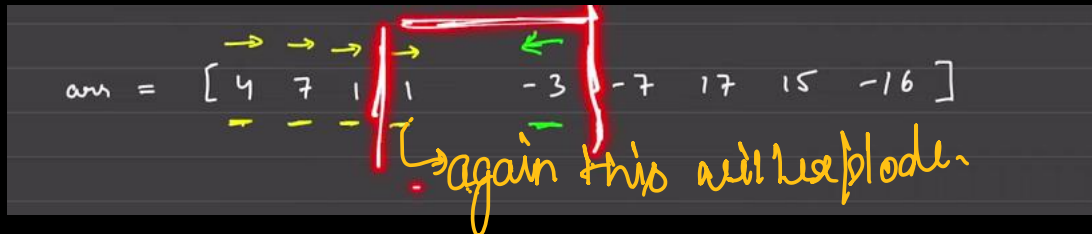
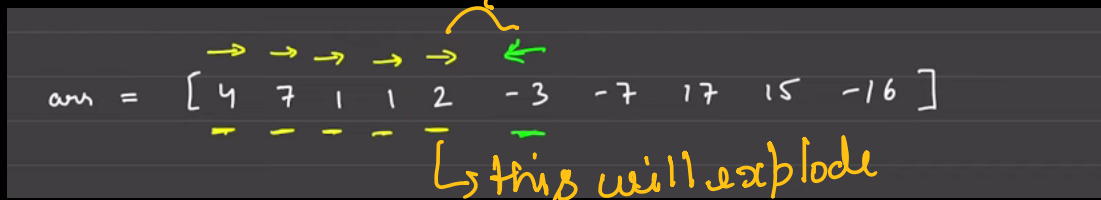
Hint

We are given an array `asteroids` of integers representing asteroids in a row. The indices of the asteroid in the array represent their relative position in space.

For each asteroid, the absolute value represents its size, and the sign represents its direction (positive meaning right, negative meaning left). Each asteroid moves at the same speed.

Find out the state of the asteroids after all collisions. If two asteroids meet, the smaller one will explode. If both are the same size, both will explode. Two asteroids moving in the same direction will never meet.

negative sign
means asteroid
is moving
in the negative
direction.



```

3     bool isEigibleForCollision(int x,int y){
4         return (x>0 && y<0);
5     }
6 public:
7     vector<int> asteroidCollision(vector<int>& asteroids) {
8         int n = asteroids.size();
9         stack<int>st;
10        st.push(asteroids[0]);
11        for(int i = 1; i<n; i++){
12            bool sign = false;
13            while(!st.empty() && abs(st.top())<=abs(asteroids[i]) && isEigibleForCollision(st.top(),
14            asteroids[i])){
15                if(abs(st.top())==abs(asteroids[i])){
16                    st.pop();
17                    sign = true;
18                    break;
19                }
20                else{
21                    st.pop();
22                }
23            }
24            if(st.empty() && !sign || !st.empty() && !isEigibleForCollision(st.top(),asteroids
25            [i]) && !sign){
26                st.push(asteroids[i]);
27            }
28        }
29        vector<int>ans;
30        while(!st.empty()){

```

Ques- Largest Area of a rectangle in a Histogram



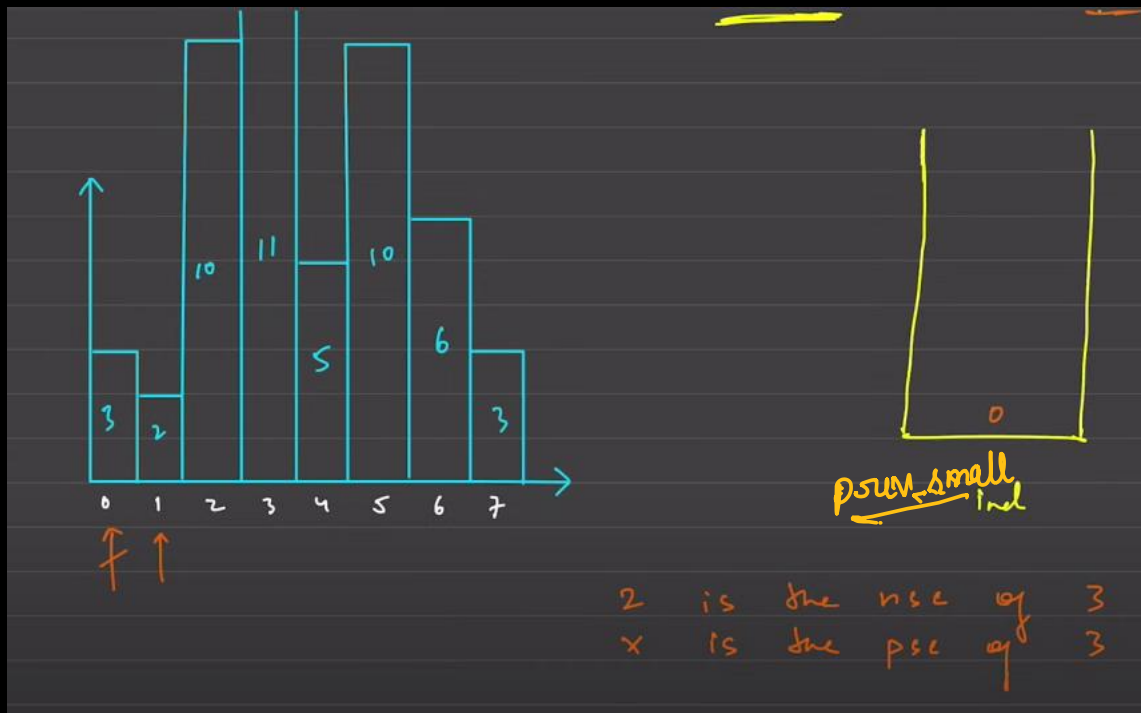
So, we need previous smaller element, & next smaller element

Approach-1 - using prev_small & next_small

While calculating the prev_small index and next_small index will use the same procedure as we have used in the sum of subarray minimums, i.e while calculating prev_small we will put -1 for the index if there is no prev_small for an element, and for there is not next small then put n-> the size of the array, for that element.

$$\text{Area} = \text{arr}[i] * (\text{next_small}[i] - \text{prev_small}[i] - 1);$$

Approach 2 -



From the stack if 0th index is popped (while calculating prev_small) it means we have got the lesser value, so for the 0th index next_small is 1st index. As after removing 0th index there is no any element in the stack so, there is no prev_small of 3. So, for first index we can calculate the rectangle area. Likely we will calculate the area of the rectangle's while popping them out of the stack.

```

C++ Auto
int largestRectangleArea(vector<int>& arr) {
    int n = arr.size();
    stack<int> st;
    int max_area = 0;
    st.push(0);
    // prev_small array----->
    for(int i = 1; i < n; i++){
        if(!st.empty() && arr[i] >= arr[st.top()]){
            st.push(i);
        }
        else{
            while(!st.empty() && arr[st.top()] > arr[i]){
                int next_small = i;
                int ele = arr[st.top()];
                st.pop();
                int prev_small = st.empty() ? -1 : st.top();
                max_area = max(max_area, ele * (next_small - prev_small - 1));
            }
            st.push(i);
        }
    }
    while(!st.empty()){
        int ele = arr[st.top()];
        st.pop();
        int prev = st.empty() ? -1 : st.top();
        int next = n;
        max_area = max(max_area, ele * (next - prev - 1));
    }
    return max_area;
}

```

Ques

85. Maximal Rectangle

Hard Topics Companies

Given a `rows x cols` binary matrix filled with 0's and 1's, find the largest rectangle containing only 1's and return its area.

Example 1:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1

1	0	1	0	1
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

1	0	1	0	1
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

1	0	1	0	1
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

→ histograms
Now find
max area of
rectangle.

1	0	1	0	1
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

we will use prefix sum matrix to make histogram -

→
prefix
sum

1	0	1	0	1
2	0	2	1	2
3	1	3	2	3
4	0	0	3	0

→ pass each
array
in prev
question

```

35 int maximalRectangle(vector<vector<char>>& matrix) {
36     int rows = matrix.size();
37     int cols = matrix[0].size();
38     vector<vector<int>> prefixsum(rows, vector<int>(cols));
39     for(int j = 0; j < cols; j++){
40         for(int i = 0; i < rows; i++){
41             if(matrix[i][j] == '1'){
42                 if(i > 0){
43                     prefixsum[i][j] = prefixsum[i-1][j] + 1;
44                 }
45                 else{
46                     prefixsum[i][j] = 1;
47                 }
48             }
49             else{
50                 prefixsum[i][j] = 0;
51             }
52         }
53     }
54     int maxisum = 0;
55     for(int i = 0; i < rows; i++){
56         vector<int> arr;
57         for(int j = 0; j < cols; j++){
58             arr.push_back(prefixsum[i][j]);
59         }
60         maxisum = max(maxisum, largestRectangleArea(arr));
61     }
62     return maxisum;

```


402. Remove K Digits

Medium

Topics

Companies

Given string `num` representing a non-negative integer `num`, and an integer `k`, return the smallest possible integer after removing `k` digits from `num`.

Example 1:

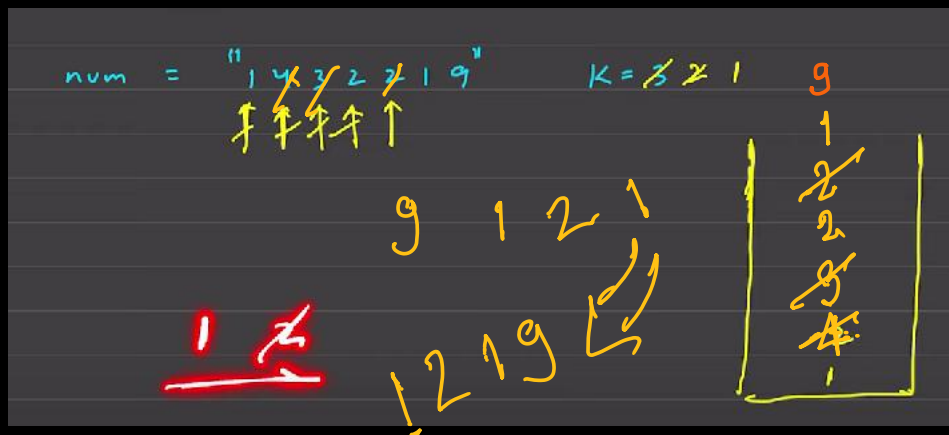
Input: `num = "1432219", k = 3`

Output: `"1219"`

Explanation: Remove the three digits 4, 3, and 2 to form the new number 1219 which is the smallest.

We will insert the elements in the stack, when a value greater than the `st.top()` comes then we push it, when lesser than `st.top()` value comes, we first `pop()` until get `st.top() < num[i]` && `k > 0` then push `num[i]`.

Once `k` becomes zero, don't stop the traversal continuing pushing the element once all remaining elements gets pushed, pop them and reverse them to get the ans.



```

2 public:
3 string removeKdigits(string num, int k) {
4     int n = num.length();
5     stack<char> st;
6     st.push(num[0]);
7     for (int i = 1; i < n; i++) {
8         if (!st.empty() && num[i] >= st.top()) {
9             st.push(num[i]);
10        } else {
11            while (!st.empty() && num[i] < st.top() && k > 0) {
12                st.pop();
13                k--;
14            }
15            st.push(num[i]);
16        }
17    }
18    while (!st.empty() && k > 0) {
19        st.pop();
20        k--;
21    }
22    string s = "";
23    while (!st.empty()) {
24        s += st.top();
25        st.pop();
26    }
27    reverse(s.begin(), s.end());
28    int idx = 0;
29    while (idx < s.size() && s[idx] == '0') idx++;
30    s = s.substr(idx);
31    return s.empty() ? "0" : s;
32 }

```

Ques. - Stock span problem - The maximum consecutive

Days for which the stock was lesser than or equal to the current one.

Online stock span → maximum consecutive days
for which the stock price was
less than or equal to current day.

StockSpanner

1	←	Next	→	7	✓
1	←	Next	→	2	✓
1	←	Next	→	1	✓
3	←	Next	→	3	✓
4	←	Next	→	3	✓
1	←	Next	→	1	✓
7	←	Next	→	8	✓

7 2 1 3 3 1 8